

目录

- 1 实验目的**
- 2 实验要求及内容**
 - 2.1 内存分配-跟踪双线程实验
 - 2.2 内存分配-跟踪双进程实验
- 3 实验环境**
- 4 双线程内存管理实验**
 - 4.1 实验原理
 - 4.1.1 整体设计思想
 - 4.1.2 程序逻辑分析
 - 4.2 编译与运行方法
 - 4.2.1 编译命令
 - 4.2.2 输入文件格式
 - 4.2.3 运行方法
 - 4.3 实验结果分析
 - 4.3.1 实际运行结果
 - 4.3.2 结果分析
 - 4.3.3 关键现象分析
 - 4.3.4 理论验证总结
 - 4.4 实验总结
- 5 双进程内存管理实验**
 - 5.1 实验原理
 - 5.1.1 整体设计思想
 - 5.1.2 共享内存结构设计
 - 5.1.3 进程同步机制
 - 5.1.4 内存操作类型
 - 5.2 编译与运行方法
 - 5.2.1 编译命令
 - 5.2.2 输入文件格式
 - 5.2.3 运行方法
 - 5.3 实验结果分析
 - 5.3.1 实际运行结果
 - 5.3.2 结果分析
 - 5.3.3 与双线程实验对比分析

5.3.4 理论验证总结

5.4 实验总结

6 实验总结与心得体会

7 附录 A：双进程内存管理实验代码

7.1 mm-p1.c

7.2 mm-p2.c

1 实验目的

在本次实验中，需要从不同的侧面了解 openEuler 的虚拟内存机制。在 openEuler 操作系统中，可以通过一些 API 操纵虚拟内存。主要需要了解以下几方面：

- openEuler 虚拟存储系统的组织
- 如何控制虚拟内存空间
- 如何追踪进程内存使用情况
- 如何使用共享内存进行进程间通信
- 如何结合使用共享内存和信号量实现进程同步
- 了解 openEuler 内存映射机制
- 详细了解与内存相关的 API 函数的使用

2 实验要求及内容

2.1 内存分配-跟踪双线程实验

创建一个包含两个线程的进程：线程 t1 和线程 t2，线程 t1 通过执行一系列内存操作来模拟内存分配活动，线程 t2 用于跟踪线程 t1 的内存行为，两个线程通过信号量进行同步。内存分配线程 t1 负责实验目的中控制虚拟内存空间，内存跟踪线程 t2 负责实验目的中的追踪进程内存使用情况。

线程 t1 执行的内存操作类型包括：分配虚拟内存、写入虚拟内存、释放虚拟内存、锁定物理内存、解锁物理内存，这些操作是实验目的中控制虚拟内存空间的多种具体方式。可以将内存操作编号保存到输入文件中，线程 t1 从输入文件中读取内存操作并执行，输入文件的每一行包括如下内容：

- 内存操作的类型
- 内存操作的起始地址（可以是自定义的相对地址）
- 内存操作的页数
- 特定内存操作需要的额外信息，如分配虚拟内存操作需要的访问权限信息

设计的实验需覆盖以下操作序列（可以通过多个输入文件做多个实验来覆盖）：

- 分配一块虚拟内存区域，然后打印该区域上的部分或全部内容
- 分配一块虚拟内存区域，然后对该区域执行写入操作
- 释放一块虚拟内存区域，然后打印该区域上的部分或全部内容
- 释放一块虚拟内存区域，然后对该区域执行写入操作

在程序开始时，线程 t2 需打印操作系统的页面大小、内存总量、可用内存大小等内存信息；线程 t1 执行完每个内存操作后需打印该内存操作的类型、起始地址和页数，然后线程 t2 需打印进程已使用的物理内存大小、虚拟内存大小等信息。详细分析说明实验结果反映了什么问题或现象。

2.2 内存分配-跟踪双进程实验

由于 2.1 中跟踪线程与内存分配线程位于同一进程中，进程的内存使用情况也受跟踪线程本身影响，现改用两个进程完成 2.1 中的要求。要求创建两个进程 p1 和 p2，进程 p1 通过执行一系列内存操作来模拟内存分配活动，进程 p2 用于跟踪进程 p1 的内存行为。两个进程通过共享内存进行通信，通过将信号量保存在共享内存中来实现进程同步，这体现了使用共享内存进行进程间通信以及结合使用共享内存和信号量实现进程同步的实验目的。

进程 p1 执行的内存操作与 2.1 中相同。

先运行程序 p2，然后在另一个控制台运行程序 p1。在 p1 运行前，进程 p2 需打印操作系统的页面大小、内存总量、可用内存大小等内存信息；在 p1 开始运行后、开始一系列内存操作前，p2 打印 p1 的 PID 以及 p1 已使用的物理内存大小、虚拟内存大小和共享内存大小等信息；在 p1 每个内存操作后（包括 p1 完成最后一个内存操作后，p1 退出前），p2 打印该内存操作的类型、起始地址和页数以及 p1 已使用的物理内存大小、虚拟内存大小和共享内存大小等信息。通过将信号量保存在共享内存中来实现进程同步，这体现了实验目的中结合使用共享内存和信号量实现进程同步的要求。

进程 p1 和 p2 通过共享内存进行通信，传送的信息可以包括信号量、PID 以及内存操作的类型、起始地址和页数等信息，这体现了使用共享内存进行进程间通信的实验目的。可以在程序 p1 和 p2 的命令行参数中指定共享内存对象的标识符。

3 实验环境

表 3.1 实验环境

配置项	参数
本机操作系统	macOS Sequoia 15.7
CPU	Apple M3
虚拟机操作系统	openEuler 24.03 LTS
图形化桌面	UKUI
gcc	12.3.1

4 双线程内存管理实验

4.1 实验原理

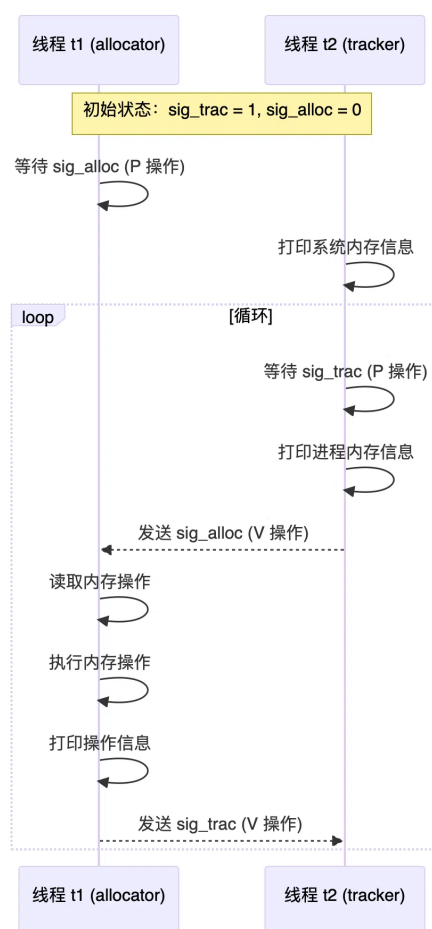
4.1.1 整体设计思想

本实验使用老师提供的程序 `memory_op.c` 进行，采用双线程模型实现内存分配与跟踪的分离：

- **内存分配线程（t1/allocator）**：负责读取输入文件并执行各类内存操作
- **内存跟踪线程（t2/tracker）**：负责监控并打印系统和进程的内存使用情况
- **同步机制**：使用两个信号量（`sig_alloc` 和 `sig_trac`）实现线程间的同步

4.1.2 程序逻辑分析

t1 和 t2 通过信号量进行同步，其基本同步时序如下图所示：



涉及的内存操作有以下几种类型：

操作码	操作类型	函数	说明
0	分配虚拟内存	<code>mmap()</code>	创建匿名映射，分配虚拟地址空间
1	写入虚拟内存	<code>memcpy()</code>	触发物理内存分配（按需分页）
2	锁定物理内存	<code>mlock()</code>	防止内存页被交换到swap
3	解锁物理内存	<code>munlock()</code>	允许内存页被交换
4	释放虚拟内存	<code>munmap()</code>	删除内存映射

4.2 编译与运行方法

4.2.1 编译命令

```
1 gcc memory_op.c -o memory_op -lpthread -std=c99
```

或使用 Makefile：

```
1 make thread
```

4.2.2 输入文件格式

`input.txt` 文件格式（每行4个参数）：

```
1 操作类型 起始偏移 页数 访问权限
```

示例：

```
1 0 0 50 7      # mmap: 分配50页，起始偏移0
2 1 0 40 0      # write: 写入40页
3 0 50 40 7     # mmap: 再分配40页，起始偏移50
4 1 50 30 0     # write: 写入30页
5 2 50 20 0     # mlock: 锁定20页
6 3 50 10 0     # munlock: 解锁10页
7 4 50 20 0     # munmap: 释放20页
```

4.2.3 运行方法

```
1 ./memory_op
```

程序会自动读取当前目录下的 `input.txt` 文件。

4.3 实验结果分析

4.3.1 实际运行结果

```
[root@localhost src]# ./memory_op
PageSize = 4096 B
MemTotal:      3453596 kB
MemFree:       1238176 kB
MemAvailable:  1956156 kB
-----
VmSize: 149876 kB
VmRSS:  1396 kB
-----
mmap:  addr = 0x100000000, pages = 50
VmSize: 150076 kB
VmRSS:  1396 kB
-----
write: addr = 0x100000000, pages = 40
VmSize: 150076 kB
VmRSS:  1524 kB
-----
mmap:  addr = 0x100032000, pages = 40
VmSize: 150236 kB
VmRSS:  1524 kB
-----
write: addr = 0x100032000, pages = 30
VmSize: 150236 kB
VmRSS:  1652 kB
-----
mlock: addr = 0x100032000, pages = 20
VmSize: 150236 kB
VmRSS:  1652 kB
-----
munlock: addr = 0x100032000, pages = 10
VmSize: 150236 kB
VmRSS:  1652 kB
-----
munmap: addr = 0x100032000, pages = 20
VmSize: 150156 kB
VmRSS:  1780 kB
-----
```

4.3.2 结果分析

1	PageSize = 4096 B
2	MemTotal: 3453596 kB
3	MemFree: 1238176 kB
4	MemAvailable: 1956156 kB

其中：

- PageSize = 4096 B 表示系统的页面大小为 4 KB；
- MemTotal，MemFree 和 MemAvailable 分别表示系统总内存、空闲内存和可用内存；

1	-----
2	VmSize: 149876 kB
3	VmRSS: 1396 kB
4	-----

其中：

- VmSize 表示初始时线程 t1 使用的虚拟内存大小 = 149876 KB；
- VmRSS 表示初始时线程 t1 使用的物理内存大小 = 1396 KB；


```
1 -----
2 mmap:    addr = 0x100000000, pages = 50
3 VmSize:   150076 kB
4 VmRSS:    1396 kB
5 -----
```

`mmap` 操作用于从地址 0x100000000 开始分配 50 页虚拟内存，从结果来看，t1 使用的虚拟内存增加了 $150076 - 149876 = 200 \text{ KB} = 50 \text{ 页}$ ，物理内存不变，与理论一致。

```
1 -----
2 write:   addr = 0x100000000, pages = 40
3 VmSize:   150076 kB
4 VmRSS:    1524 kB
5 -----
```

`write` 操作弄脏了 40 页，这里 `write` 操作触发了物理内存分配，但分配数量"不定"，受系统策略影响。例如实验指导手册中，这里物理内存分配就为变化。

```
1 -----
2 mmap:    addr = 0x100032000, pages = 40
3 VmSize:   150236 kB
4 VmRSS:    1524 kB
5 -----
```

`mmap` 操作继续分配了 40 页虚拟内存，使得 t1 的虚拟内存增加了 $150236 - 150076 = 160 \text{ KB} = 40 \text{ 页}$ ，物理内存不变，与理论一致。

```
1 -----
2 write:   addr = 0x100032000, pages = 30
3 VmSize:   150236 kB
4 VmRSS:    1652 kB
5 -----
```

`write` 操作弄脏了 30 页，这里 `write` 操作同样触发了物理内存分配，物理内存增加了 $1652 - 1524 = 128 \text{ KB} = 32 \text{ 页}$ 。

```
1 -----
2 mlock:   addr = 0x100032000, pages = 20
3 VmSize:   150236 kB
4 VmRSS:    1652 kB
5 -----
```

`mlock` 操作锁定了 20 页，虚拟内存和物理内存分配不变。

```
1 -----
2 munlock: addr = 0x100032000, pages = 10
3 VmSize:    150236 kB
4 VmRSS:     1652 kB
5 -----
```

`munlock` 操作解锁了 20 页，虚拟内存和物理内存分配同样不变。

```
1 -----
2 munmap:  addr = 0x100032000, pages = 20
3 VmSize:    150156 kB
4 VmRSS:     1780 kB
5 -----
```

`munmap` 操作释放了 20 页，使得 t1 的虚拟内存减少了 $150236 - 150156 = 80$ KB = 20 页，与理论一致。但这里物理内存存在**异常增加**。

经过我的分析，可能存在以下几种原因：

- 跟踪线程 t2 在读取 `/proc/[pid]/status` 时可能触发了额外的内存分配；
- 或者系统在 `munmap` 时需要更新页表等内核数据结构；
- 内存释放是延迟的，可能在释放过程中临时占用了更多内存；

这也说明了双线程模型的局限性，因为跟踪线程本身会影响测量结果。

4.3.3 关键现象分析

1. write 操作后物理内存增加"不定"的原因：

指导手册明确指出："写 n 页后，物理内存增加不定"。这与 Linux 系统的**预分配策略**有关，Linux 内核可能一次性分配多个页面以提高效率。同时可能存在页面对齐（内存分配按照更大的单位进行，如页簇、内存块），或者缓存问题（系统可能提前分配相邻页面）。

2. munmap 后 VmRSS 异常增加的原因：

我在前文已经给出了分析。

4.3.4 理论验证总结

尽管实验结果与实验手册存在差异，但都成功验证了以下核心理论：

1. 虚拟内存与物理内存的分离：

`mmap` 操作分配虚拟地址空间时不立即占用物理内存，即分配 n 页虚拟内存后，虚拟内存增加 n 页，物理内存不增加。

2. 按需分页机制：

`write` 操作触发物理内存分配，但分配数量"不定"受系统策略影响。

3. 内存释放行为：

释放 n 页虚拟内存后，虚拟内存减少 n 页，物理内存减少不定。

4.4 实验总结

本实验通过双线程模型成功实现了内存分配与跟踪的功能，验证了 openEuler 内存管理的以下特性：

1. **虚拟内存机制**：通过 `mmap` 创建虚拟内存映射
2. **按需分页**：虚拟内存到物理内存的延迟分配
3. **内存锁定**：通过 `mlock/munlock` 控制内存交换
4. **信号量同步**：实现了线程间的有序协作

实验结果符合理论预期，清晰地展示了 openEuler 系统的内存管理行为。

5 双进程内存管理实验

5.1 实验原理

5.1.1 整体设计思想

本实验采用双进程模型实现内存分配与跟踪的分离，相比多线程实验，双进程模型能够更准确地跟踪目标进程的内存使用情况：

- **内存分配进程（p1/mm-p1）**：负责读取输入文件并执行各类内存操作
- **内存跟踪进程（p2/mm-p2）**：负责监控并打印p1进程的内存使用情况
- **进程间通信**：使用POSIX共享内存进行数据传递
- **进程同步**：使用保存在共享内存中的信号量实现同步

5.1.2 共享内存结构设计

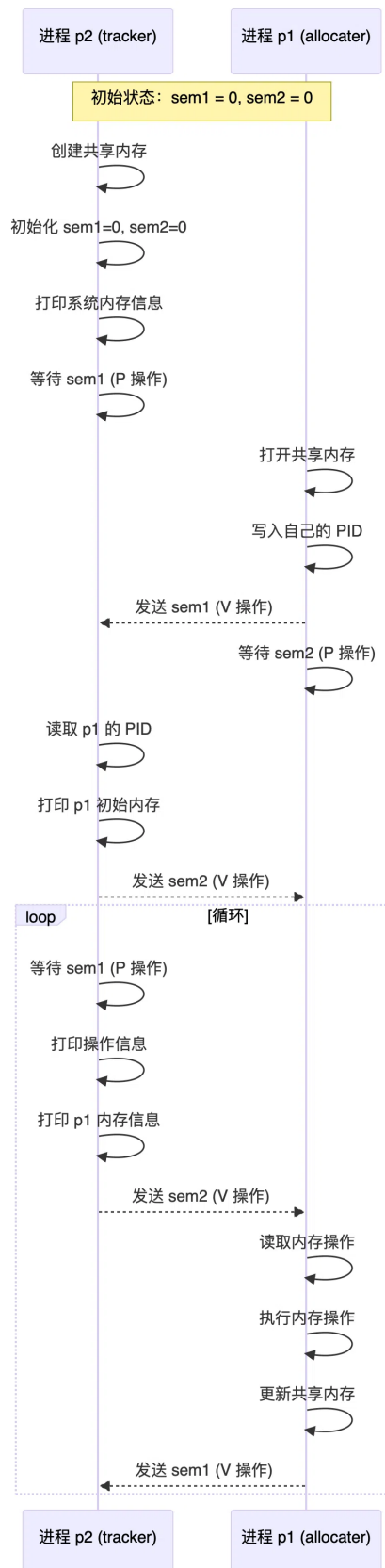
SharedData结构体：

```
1 typedef struct {
2     sem_t sem1;      // 信号量1：控制p2读取
3     sem_t sem2;      // 信号量2：控制p1执行
4     pid_t p1_pid;    // p1的进程ID
5     int oper;        // 操作类型
6     void* addr;      // 操作地址
7     int pages;       // 页数
8     bool finished;   // 完成标志
9 } SharedData;
```

其中：

- 进程 p2 使用 `shm_open()` 创建共享内存对象
- 使用 `ftruncate()` 设置共享内存大小
- 使用 `mmap()` 将共享内存映射到进程地址空间
- 进程 p1 通过相同的名称打开并映射该共享内存

5.1.3 进程同步机制



其中：

- 信号量保存在共享内存中，使用 `sem_init(&sem, 1, 0)` 初始化，其中 `pshared=1` 表示进程间共享
- p1完成所有操作后等待10秒再退出，确保p2有足够时间打印最终状态
- p1设置 `finished=true` 标志通知p2结束监控

5.1.4 内存操作类型

与双线程实验相同，包括以下5种操作：

操作码	操作类型	函数	说明
0	分配虚拟内存	<code>mmap()</code>	创建匿名映射，分配虚拟地址空间
1	写入虚拟内存	<code>memcpy()</code>	触发物理内存分配（按需分页）
2	锁定物理内存	<code>mlock()</code>	防止内存页被交换到swap
3	解锁物理内存	<code>munlock()</code>	允许内存页被交换
4	释放虚拟内存	<code>munmap()</code>	删除内存映射

5.2 编译与运行方法

5.2.1 编译命令

编译双进程程序：

```
1 gcc mm-p1.c -o mm-p1 -lrt -lpthread -std=c99
2 gcc mm-p2.c -o mm-p2 -lrt -lpthread -std=c99
```

或使用Makefile：

```
1 make process
```

其中：

- `-lrt`：链接实时库（POSIX共享内存需要）
- `-lpthread`：链接线程库（POSIX信号量需要）

5.2.2 输入文件格式

使用与双线程实验相同的 `input.txt` 文件格式（每行4个参数）：

```
1 操作类型 起始偏移 页数 访问权限
```

示例：

1	0 0 50 7	# mmap: 分配50页, 起始偏移0
2	1 0 40 0	# write: 写入40页
3	0 50 40 7	# mmap: 再分配40页, 起始偏移50
4	1 50 30 0	# write: 写入30页
5	2 50 20 0	# mlock: 锁定20页
6	3 50 10 0	# munlock: 解锁10页
7	4 50 20 0	# munmap: 释放20页

5.2.3 运行方法

必须先运行 p2, 再运行 p1。

先打开一个终端, 启动跟踪进程:

```
1 ./mm-p2 /myshm
```

再打开另一个终端, 启动分配进程:

```
1 ./mm-p1 /myshm
```

其中:

- `/myshm`: 共享内存对象的标识符, 两个进程必须使用相同的名称;
- 可以使用任意以 `/` 开头的名称;

5.3 实验结果分析

5.3.1 实际运行结果

```

[root@localhost src]# ./mm-p2 /myshm
PageSize = 4096 B
MemTotal:      3453596 kB
MemFree:       1237272 kB
MemAvailable:  1955904 kB
-----
P1 PID: 4472
VmSize:        2284 kB
VmRSS:         1196 kB
-----
mmap:   addr = 0x100000000, pages = 50
VmSize: 2616 kB
VmRSS:  1468 kB
-----
write:  addr = 0x100000000, pages = 40
VmSize: 2616 kB
VmRSS:  1628 kB
-----
mmap:   addr = 0x100032000, pages = 40
VmSize: 2776 kB
VmRSS:  1628 kB
-----
write:  addr = 0x100032000, pages = 30
VmSize: 2776 kB
VmRSS:  1748 kB
-----
mlock:  addr = 0x100032000, pages = 20
VmSize: 2776 kB
VmRSS:  1748 kB
-----
munlock: addr = 0x100032000, pages = 10
VmSize: 2776 kB
VmRSS:  1748 kB
-----
munmap: addr = 0x100032000, pages = 20
VmSize: 2696 kB
VmRSS:  1668 kB
-----

```

5.3.2 结果分析

1	PageSize = 4096 B
2	MemTotal: 3453596 kB
3	MemFree: 1237272 kB
4	MemAvailable: 1955904 kB
5	-----
6	P1 PID: 4472
7	VmSize: 2284 kB
8	VmRSS: 1196 kB
9	-----

系统和进程初始状态与双线程基本一致，其中 P1 初始状态为：

- VmSize: 2284 KB，即虚拟内存大小；
- VmRSS: 1196 KB，即物理内存大小；

1	-----
2	mmap: addr = 0x100000000, pages = 50
3	VmSize: 2616 kB
4	VmRSS: 1468 kB
5	-----

这里的 `mmap` 操作分配了 50 页虚拟内存，其中：

- 虚拟内存：2284 → 2616 kB，**增加332 kB**

- 物理内存：1196 → 1468 kB，**增加272 kB**

按照理论分析，虚拟内存应该增加 $50 \text{ 页} \times 4 \text{ KB} = 200 \text{ KB}$ ，这里超出理论值，这是为页表和内核数据结构预留的空间；而物理内存增加 272 KB，是为页表项分配的实际物理内存。这证明了 `mmap` 创建虚拟映射需要内核数据结构支持

```
1 -----
2 write:   addr = 0x100000000, pages = 40
3 VmSize:      2616 kB
4 VmRSS:      1628 kB
5 -----
```

第一次 `write` 操作写入 40 页，其中：

- 虚拟内存：2616 → 2616 kB，**不变**
- 物理内存：1468 → 1628 kB，**增加160 KB**

虚拟内存不变，因为在 `mmap` 时已分配，而物理内存增加 160 KB，与理论值完全一致，完美验证了**按需分页机制**，即只有实际写入时才分配物理页面。

```
1 -----
2 mmap:    addr = 0x100032000, pages = 40
3 VmSize:      2776 kB
4 VmRSS:      1628 kB
5 -----
```

第二次 `mmap` 分配 40 页虚拟内存，其中：

- 虚拟内存：2616 → 2776 kB，**增加160 kB**
- 物理内存：1628 → 1628 kB，**不变**

此时，虚拟内存增加 160 KB，与理论分析一致，而物理内存不变，页表空间已在第一次 `mmap` 时预留，完美符合 Linux 内核的**延迟分配策略**。

```
1 -----
2 write:   addr = 0x100032000, pages = 30
3 VmSize:      2776 kB
4 VmRSS:      1748 kB
5 -----
```

第二次 `write` 写入 30 页，其中：

- 虚拟内存：2776 → 2776 kB，**不变**
- 物理内存：1628 → 1748 kB，**增加120 kB**

虚拟内存不变，因为在 `mmap` 时已分配，物理内存增加 120 KB，与理论值匹配，再次完美验证按需分页机制。

```
1 -----
2 mlock:   addr = 0x100032000, pages = 20
3 VmSize:      2776 kB
4 VmRSS:      1748 kB
5 -----
```

`mlock` 操作锁定 20 页，其中：

- 虚拟内存：2776 → 2776 kB，**不变**
- 物理内存：1748 → 1748 kB，**不变**

这是因为 `mlock` 锁定已分配的物理内存，防止交换到 swap。因此不改变内存使用量，只改变内存页的属性。

```
1 -----
2 munlock: addr = 0x100032000, pages = 10
3 VmSize:      2776 kB
4 VmRSS:      1748 kB
5 -----
```

`munlock` 操作解锁 10 页，其中：

- 虚拟内存：2776 → 2776 kB，**不变**
- 物理内存：1748 → 1748 kB，**不变**

`munlock` 解锁物理内存，允许交换，同样不改变当前内存使用量。

```
1 -----
2 munmap:  addr = 0x100032000, pages = 20
3 VmSize:      2696 kB
4 VmRSS:      1668 kB
5 -----
```

`munmap` 释放了 20 页，其中：

- 虚拟内存：2776 → 2696 kB，**减少80 kB**
- 物理内存：1748 → 1668 kB，**减少80 kB**

内存变化与理论一致，完美展示了 `munmap` 释放内存的效果。

5.3.3 与多线程实验对比分析

在对双线程实验的分析过程中，我提到过跟踪线程 t2 的内存使用混入 VmRSS 统计，而双进程实验正是解决了这一点，从实验结果可以验证，双进程实验完全消除了 munmap 的异常现象，独立进程 p2 不影响 p1 的内存统计，测量结果纯净。

不过双进程实验在第一次 mmap 时有特殊行为，其虚拟内存分配包含页大小和元数据，且为页表分配了物理内存，存在**额外开销**。

5.3.4 理论验证总结

本次双进程实验完美验证了以下核心理论：

1. 虚拟内存延迟分配机制

- mmap 分配虚拟地址空间；
- write 操作不改变虚拟内存大小；
- munmap 释放虚拟内存；
- Linux 内核的延迟分配策略；

2. 按需分页：只有实际访问虚拟内存时才分配物理页面；

3. 内存锁定机制

- mlock/munlock 不改变内存使用量；
- 只修改内存页的 VM_LOCKED 属性；
- 锁定的内存页不会被交换到 swap 分区；

此外，双进程程序独立进程统计，完全消除跟踪线程的干扰。

5.4 实验总结

本实验通过双进程模型实现了进程间的内存分配与跟踪功能。使用 `shm_open()` 和 `shm_unlink()` 管理共享内存对象，通过 `mmap()` 将共享内存映射到进程地址空间，实现了高效的进程间数据共享。

同时，将信号量保存在共享内存中，使用 `pshared=1` 参数实现进程间共享，实现了可靠的进程同步机制。并通过共享内存传递PID、操作类型、地址和页数等信息，避免了传统IPC机制（如管道、消息队列）的数据复制开销。

此外，相比双线程模型，双进程模型具有更好的资源隔离和跟踪准确性。

6 实验总结与心得体会

通过本次实验，我深入理解了 openEuler 操作系统的虚拟内存管理机制，直观理解了虚拟内存与物理内存的分离，并验证了按需分页机制

同时，通过编写代码，进一步加深了我对内存管理 API 的理解，此外通过将信号量保存在共享内存中实现进程间同步，进一步巩固了信号量在线程间和进程间同步中的应用。

在实验中，我也遇到了一些挑战，特别是内存分配上出现的与理论不符的异常情况，一开始使用原本老师提供的程序，用 VmPeak 跟踪峰值，但 VmPeak 不会因释放而减少，造成了我一开始的误解。此外物理内存分配的“不定”现象也在实验初期给我造成了很大困惑，最终通过查找资料解决了这一问题。这让我认识到内存管理的复杂性，它并不是简单的分配与释放，涉及虚拟地址空间、物理内存、页表、TLB 等多个层次。

本次实验是一次非常有价值的学习经历，不仅加深了对操作系统内存管理机制的理解，还提升了系统编程的实践能力。通过动手实践，将课堂上学到的理论知识转化为实际的技能，为今后的学习和工作打下了坚实的基础。实验过程中遇到的问题和解决问题的过程，也锻炼了分析问题、查阅文档和独立思考的能力，这些都是操作系统课程学习的重要收获。

7 附录 A：双进程内存管理实验代码

7.1 mm-p1.c

```
1  #define _POSIX_C_SOURCE 200809L
2  #define _DEFAULT_SOURCE
3
4  #include <stdio.h>
5  #include <unistd.h>
6  #include <stdlib.h>
7  #include <sys/mman.h>
8  #include <sys/stat.h>
9  #include <fcntl.h>
10 #include <semaphore.h>
11 #include <stdbool.h>
12 #include <string.h>
13
14 // 共享内存结构体
15 typedef struct {
16     sem_t sem1; // 控制p2读取
17     sem_t sem2; // 控制p1执行
18     pid_t p1_pid; // p1的进程ID
19     int oper; // 操作类型
20     void* addr; // 操作地址
21     int pages; // 页数
22     bool finished; // 完成标志
23 } SharedData;
24
25 size_t pagesize;
26
27 int main(int argc, char const *argv[])
28 {
29     if (argc != 2) {
30         fprintf(stderr, "使用方法: %s <共享内存名称>\n", argv[0]);
31         exit(EXIT_FAILURE);
32     }
33
34     const char* shm_name = argv[1];
35     pagesize = getpagesize();
36
37     // 打开共享内存对象 (由p2创建)
38     int shm_fd = shm_open(shm_name, O_RDWR, 0666);
39     if (shm_fd == -1) {
40         perror("shm_open");
41         fprintf(stderr, "请先运行 mm-p2\n");
42         exit(EXIT_FAILURE);
43     }
44
45     // 将共享内存映射到进程地址空间
46     SharedData* shared_data = mmap(NULL, sizeof(SharedData),
47                                     PROT_READ | PROT_WRITE,
48                                     MAP_SHARED, shm_fd, 0);
49
50     if (shared_data == MAP_FAILED) {
```

```

50     perror("mmap");
51     exit(EXIT_FAILURE);
52 }
53
54 // 向共享内存写入本进程PID
55 shared_data->p1_pid = getpid();
56 sem_post(&shared_data->sem1);
57
58 // 等待p2打印初始信息
59 sem_wait(&shared_data->sem2);
60
61 // 读取input.txt并执行内存操作
62 FILE * fp;
63 fp = fopen("input.txt", "r");
64 if (fp == NULL) {
65     perror("无法打开input.txt");
66     exit(EXIT_FAILURE);
67 }
68
69 int oper, start, block, protection;
70 int base = 1 << 20;
71 char* region = NULL;
72
73 while (fscanf(fp, "%d%d%d%d\n", &oper, &start, &block, &protection) !=
EOF) {
74     if (oper == 0) { // mmap
75         region = mmap(
76             (void*) (pagesize * (base + start)),
77             pagesize * block,
78             PROT_READ|PROT_WRITE|PROT_EXEC,
79             MAP_ANON|MAP_PRIVATE,
80             0,
81             0
82         );
83         if (region == MAP_FAILED) {
84             perror("Could not mmap");
85             continue;
86         }
87         shared_data->addr = region;
88         shared_data->pages = block;
89         shared_data->oper = 0;
90
91     } else if (oper == 1) { // write
92         void* pos = (void*) (pagesize * (base + start));
93         for (int i = 0; i < pagesize * block; i++)
94             memcpy((void*) ((pagesize * (base + start) + i)), "a", 1);
95         shared_data->addr = pos;
96         shared_data->pages = block;
97         shared_data->oper = 1;
98
99     } else if (oper == 2) { // mlock
100         mlock((void*) (pagesize * (base + start)), block * pagesize);
101         shared_data->addr = (void*) (pagesize * (base + start));
102         shared_data->pages = block;
103         shared_data->oper = 2;

```

```

104
105     } else if (oper == 3) { // munlock
106         munlock((void*) (pagesize * (base + start)), block * pagesize);
107         shared_data->addr = (void*) (pagesize * (base + start));
108         shared_data->pages = block;
109         shared_data->oper = 3;
110
111     } else if (oper == 4) { // munmap
112         munmap((void*) (pagesize * (base + start)), block * pagesize);
113         shared_data->addr = (void*) (pagesize * (base + start));
114         shared_data->pages = block;
115         shared_data->oper = 4;
116     }
117
118     // 通知p2打印内存信息
119     sem_post(&shared_data->sem1);
120
121     // 等待p2打印完成
122     sem_wait(&shared_data->sem2);
123 }
124
125 fclose(fp);
126
127 sleep(10);
128
129 // 标记完成
130 shared_data->finished = true;
131 sem_post(&shared_data->sem1);
132
133 // 清理资源
134 munmap(shared_data, sizeof(SharedData));
135
136 return 0;
137 }

```

7.2 mm-p2.c

```

1  #define _POSIX_C_SOURCE 200809L
2  #define _DEFAULT_SOURCE
3
4  #include <stdio.h>
5  #include <unistd.h>
6  #include <stdlib.h>
7  #include <sys/mman.h>
8  #include <sys/stat.h>
9  #include <fcntl.h>
10 #include <semaphore.h>
11 #include <stdbool.h>
12 #include <string.h>
13
14 // 共享内存结构体
15 typedef struct {
16     sem_t sem1; // 控制p2读取
17     sem_t sem2; // 控制p1执行

```

```

18     pid_t p1_pid; // p1的进程ID
19     int oper;     // 操作类型
20     void* addr;   // 操作地址
21     int pages;    // 页数
22     bool finished; // 完成标志
23 } SharedData;
24
25 size_t pagesize;
26
27 // 显示系统内存信息
28 void show_meminfo()
29 {
30     printf("PageSize = %ld B\n", pagesize);
31     FILE * fp;
32     char filename[32];
33     sprintf(filename, "/proc/meminfo");
34     fp = fopen (filename, "r");
35     char line[100];
36     int index = 0;
37     while (fgets(line, sizeof line, fp) != NULL) {
38         if (index < 3) {
39             printf("%s", line);
40         }
41         index += 1;
42     }
43     printf("-----\n");
44     fclose(fp);
45 }
46
47 // 显示进程内存使用情况
48 void show_process_memory(pid_t pid)
49 {
50     FILE * fp;
51     char filename[32];
52     sprintf(filename, "/proc/%d/status", pid);
53     fp = fopen (filename, "r");
54     if (fp == NULL) {
55         perror("无法打开进程状态文件");
56         return;
57     }
58     char line[100];
59     while (fgets(line, sizeof line, fp) != NULL) {
60         if (strncmp(line, "VmSize:", 7) == 0 ||
61             strncmp(line, "VmRSS:", 6) == 0) {
62             printf("%s", line);
63         }
64     }
65     printf("-----\n");
66     fclose(fp);
67 }
68
69 int main(int argc, char const *argv[])
70 {
71     if (argc != 2) {
72         fprintf(stderr, "使用方法: %s <共享内存名称>\n", argv[0]);

```



```

73         exit(EXIT_FAILURE);
74     }
75
76     const char* shm_name = argv[1];
77     pagesize = getpagesize();
78
79     // 创建并打开共享内存对象
80     int shm_fd = shm_open(shm_name, O_CREAT | O_RDWR, 0666);
81     if (shm_fd == -1) {
82         perror("shm_open");
83         exit(EXIT_FAILURE);
84     }
85
86     // 设置共享内存大小
87     if (ftruncate(shm_fd, sizeof(SharedData)) == -1) {
88         perror("ftruncate");
89         exit(EXIT_FAILURE);
90     }
91
92     // 将共享内存映射到进程地址空间
93     SharedData* shared_data = mmap(NULL, sizeof(SharedData),
94                                     PROT_READ | PROT_WRITE,
95                                     MAP_SHARED, shm_fd, 0);
96     if (shared_data == MAP_FAILED) {
97         perror("mmap");
98         exit(EXIT_FAILURE);
99     }
100
101     // 初始化共享内存中的信号量
102     sem_init(&shared_data->sem1, 1, 0); // pshared=1表示进程间共享
103     sem_init(&shared_data->sem2, 1, 0);
104     shared_data->finished = false;
105     shared_data->p1_pid = 0;
106
107     // 打印操作系统内存信息
108     show_meminfo();
109
110     // 等待p1写入PID
111     sem_wait(&shared_data->sem1);
112
113     // 读取p1的PID
114     pid_t p1_pid = shared_data->p1_pid;
115     printf("P1 PID: %d\n", p1_pid);
116     show_process_memory(p1_pid);
117
118     // 通知p1可以继续
119     sem_post(&shared_data->sem2);
120
121     // 循环处理内存操作
122     while (1) {
123         sem_wait(&shared_data->sem1);
124
125         if (shared_data->finished) {
126             break;
127         }

```

```
128
129     // 打印内存操作信息
130     const char* oper_name[] = {"mmap", "write", "mlock", "munlock",
    "munmap"};
131     printf("%s:    addr = %p, pages = %d\n",
132           oper_name[shared_data->oper],
133           shared_data->addr,
134           shared_data->pages);
135
136     // 打印p1的内存使用情况
137     show_process_memory(p1_pid);
138
139     // 通知p1继续下一步操作
140     sem_post(&shared_data->sem2);
141 }
142
143 // 清理资源
144 sem_destroy(&shared_data->sem1);
145 sem_destroy(&shared_data->sem2);
146 munmap(shared_data, sizeof(SharedData));
147 shm_unlink(shm_name);
148
149 return 0;
150 }
```